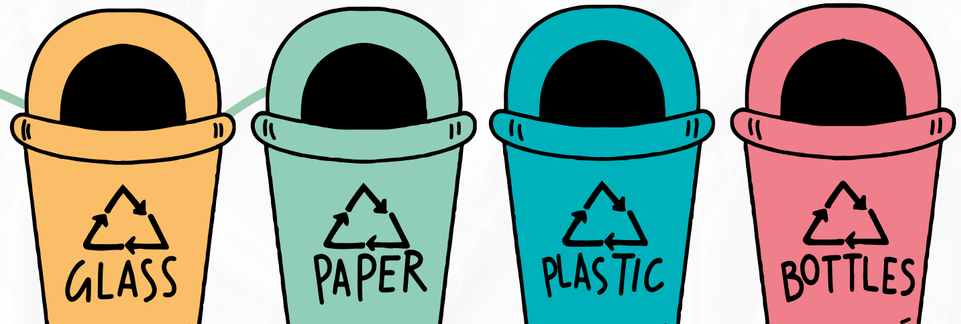
EcoSignal

Signaler une pollution, suivre son traitement : une plateforme citoyenne en microservices pour la Tunisie.



LE PROBLÈME

Les décharges sauvages sont signalées, rarement suivies

2,6 M t

de déchets ménagers par an en Tunisie

4–7 %

seulement sont recyclés

≈ 85 %

partent en décharge

Le citoyen ne sait pas si son signalement a été pris en compte. La municipalité manque de données pour décider où intervenir en premier.

Source : document de cadrage du thème, mini-projet AWD

ACTEURS

Citoyen

Rôle CITIZEN

- Signale avec photo et position GPS
- Suit l’état de ses signalements
- Reçoit des notifications
- Gagne des points de réputation

Agent municipal

Rôle MUNICIPAL_AGENT

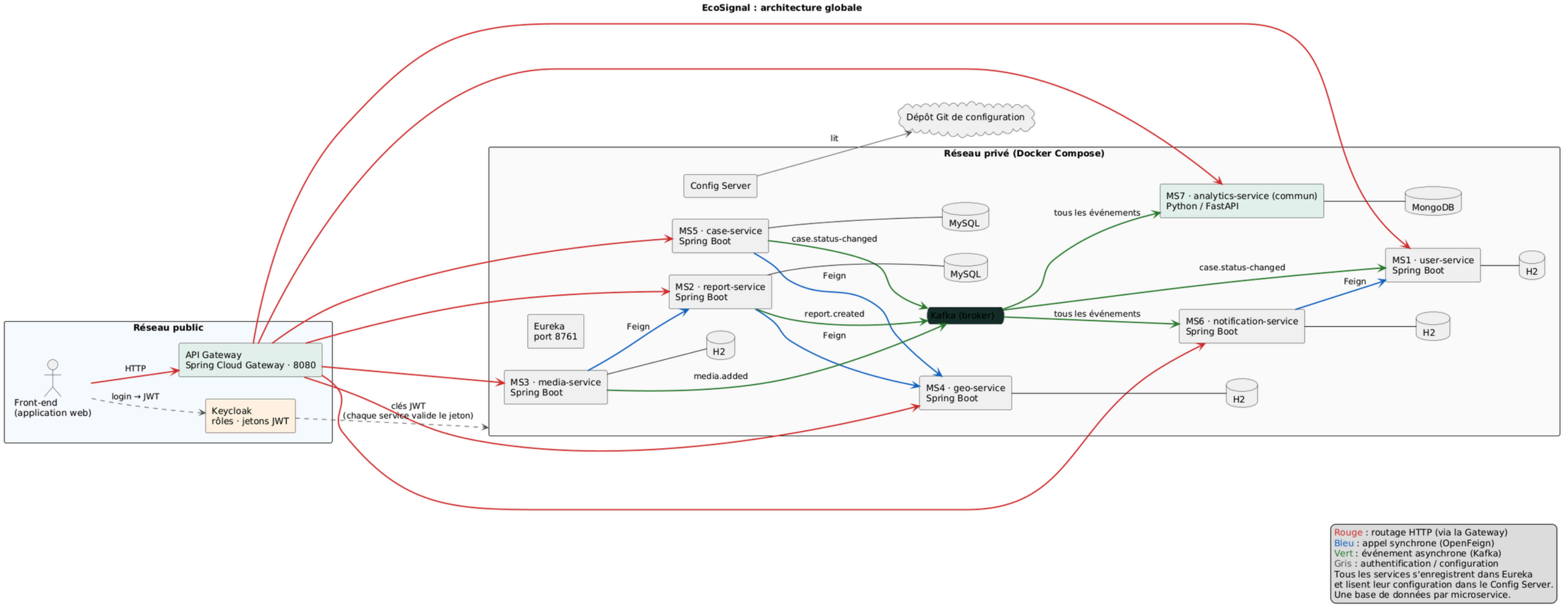
- Voit les dossiers de sa commune
- Assigne un dossier
- Change le statut
- Ajoute un commentaire

Administrateur

Rôle ADMIN

- Gère communes et zones
- Gère utilisateurs et rôles
- Consulte les statistiques
- Repère les zones les plus touchées

ARCHITECTURE GLOBALE



DÉCOUPAGE

Sept microservices, un par capacité métier

N°	Microservice	Responsabilité	Techno	Base
MS1	user-service	Profils, commune, points	Spring Boot	H2
MS2	report-service	Cycle de vie d’un signalement	Spring Boot	MySQL
MS3	media-service	Photos des signalements	Spring Boot	H2
MS4	geo-service	Communes et rattachement GPS	Spring Boot	H2
MS5	case-service	Dossiers municipaux et statuts	Spring Boot	MySQL
MS6	notification-service	Notifications et historique	Spring Boot	H2
MS7	analytics-service	Statistiques, zones à risque	Python / FastAPI	MongoDB

DONNÉES

Une base par microservice, choisie selon la donnée

MySQL

report, case

Données centrales à conserver : signalements, dossiers, historique des statuts, commentaires. Relations et filtres par statut et par date.

H2

user, media, geo, notification

Tables simples, peu de requêtes, démarrage rapide en développement. Les photos sont stockées sur un volume Docker : seules les métadonnées sont en base.

MongoDB

analytics

Événements de forme variable et agrégations par commune, catégorie et période : la technologie avancée du microservice commun.

INTERACTIONS

Synchrone quand on attend la réponse, asynchrone sinon

Synchrone (OpenFeign)

- report → geo : GPS vers commune
- media → report : le signalement existe-t-il ?
- notification → user : e-mail et préférences
- case → geo : la commune existe-t-elle ?

Asynchrone (événements)

- **report.created** : case, notification, analytics
- **case.status-changed** : report, notification, user, analytics
- **media.added** : analytics

Si un service appelé est en panne : circuit breaker et repli. Exemple : sans geo-service, le signalement est quand même enregistré (statut LOCATION_PENDING) puis relancé.

EXEMPLE DE BOUT EN BOUT

Un signalement traverse cinq services

1

Le citoyen envoie le signalement à report

2

report demande la commune à geo (synchrone)

3

report enregistre et publie report.created

4

case crée le dossier, notification informe, analytics compte

5

L’agent résout : case.status-changed, le citoyen gagne des points

TECHNOLOGIES

Composant	Technologie
Microservices MS1 à MS6	Spring Boot, API REST, Swagger
Microservice commun MS7	Python avec FastAPI
Point d’entrée	Spring Cloud Gateway
Découverte	Eureka
Sécurité	Keycloak (rôles et jetons)
Bases de données	H2, MySQL, MongoDB
Conteneurs	Docker et Docker Compose
Front-end	Application web

PLANNING ET RISQUES

Planning

- **S4** : validation de la conception
- **S5** : Gateway et Eureka
- **S6** : communication synchrone
- **S7** : événements et analytics
- **S8** : sécurité Keycloak
- **S9** : Docker et front-end
- **S10** : démonstration

Risques et parades

- **Dépendances entre services** : contrats d'API figés dès la S4, réponses simulées pour les tests
- **Intégration tardive** : une intégration partielle chaque semaine
- **Charge inégale** : revue hebdomadaire du tableau de tâches
- **Keycloak et Docker** : un binôme responsable de chacun

MS1 · AYMEN RADDEDI · SPRING BOOT · H2

user-service

Diagramme de classes

UserProfile

- id
- keycloakId
- nom
- commune
- points

Endpoint	Rôle	Codes
GET /users/me	CITIZEN	200, 404
PUT /users/me	CITIZEN	200, 400
GET /users/{id}/points	ADMIN	200, 404

Synchrone : appelé par notification-service (userId vers e-mail et préférences)
Asynchrone : consomme case.status-changed et ajoute des points si le dossier est résolu
Si panne : aucun appel sortant ; les événements attendent dans le broker

MS2 · MARWAN AZAIZ · SPRING BOOT · MYSQL

report-service

Diagramme de classes

Report

- id, userId, categorie
- description
- latitude, longitude
- communeId, date
- statut : StatutSignalement

«enum» StatutSignalement

- SUBMITTED, LOCATION_PENDING
- IN_PROGRESS, RESOLVED, REJECTED

Endpoint	Rôle	Codes
POST /reports	CITIZEN	201, 400
GET /reports?status=&communeId=	AGENT, ADMIN	200, 400
GET /reports/me	CITIZEN	200
GET /reports/{id}	tous	200, 403, 404

Synchrone : appelle geo-service (GPS vers commune) avec OpenFeign
Asynchrone : publie report.created ; consomme case.status-changed
Si panne : sans geo-service, le signalement est enregistré en LOCATION_PENDING puis relancé

MS3 · RAYEN BEN CHTILA · SPRING BOOT · H2

media-service

Diagramme de classes

Photo

- id
- reportId
- nomFichier
- type, taille
- date

Endpoint	Rôle	Codes
POST /reports/{id}/photos	CITIZEN	201, 400, 404, 413, 503
GET /photos/{id}	tous	200, 404

Synchrone : appelle report-service (le signalement existe-t-il, et qui en est l'auteur ?)
Asynchrone : publie media.added
Si panne : sans report-service, la photo est refusée (503) et le front-end réessaie ; aucun fichier orphelin

MS4 · AZIZ GASMI · SPRING BOOT · H2

geo-service

Diagramme de classes

Commune

- id
- nom
- gouvernorat
- zone géographique

Endpoint	Rôle	Codes
GET /communes	tous	200
GET /communes/locate?lat=&lon=	interne	200, 400, 404
POST /communes	ADMIN	201, 400, 409

Synchrone : répond aux appels de report-service (GPS vers commune) et de case-service
Asynchrone : aucun événement
Si panne : n’appelle personne ; ses appelants utilisent leur repli (LOCATION_PENDING pour report)

MS5 · HAMZA JLASSI · SPRING BOOT · MYSQL

case-service

Diagramme de classes

Case

- id, reportId, communeId
- agentId, statut, priorité

CaseComment

- id, caseId, texte, date

StatusHistory

- id, caseId, ancien et nouveau statut, date

Endpoint	Rôle	Codes
GET /cases?communeId=&status=	AGENT	200, 400
PUT /cases/{id}/assign	AGENT	200, 404, 409
PUT /cases/{id}/status	AGENT	200, 400, 404
POST /cases/{id}/comments	AGENT	201, 400, 404

Synchrone : appelle geo-service (la commune existe-t-elle ?)
Asynchrone : consomme report.created ; publie case.status-changed
Si panne : sans geo-service, le dossier est créé avec la commune de l'événement, sans contrôle supplémentaire

MS6 - MOHAMED ISLEM MNAKBI - SPRING BOOT - H2

notification-service

Diagramme de classes

Notification

- id
- userId
- canal
- message
- lu, date

Endpoint	Rôle	Codes
GET /notifications/me	CITIZEN	200
PUT /notifications/{id}/read	CITIZEN	204, 404

Synchrone : appelle user-service (e-mail et préférences du citoyen)
Asynchrone : consomme report.created et case.status-changed
Si panne : sans user-service, repli sur la notification dans l'application ; l'e-mail est reporté

MS7 · MICROSERVICE COMMUN · PYTHON · MONGODB

analytics-service

Modèle de données

Événement

- type, reportId
- communeId, catégorie
- statut, date

Agrégat

- commune, catégorie, période
- total

Endpoint	Rôle	Codes
GET /stats/hotspots?period=	ADMIN	200, 400
GET /stats/by-category	ADMIN	200

Synchrone : aucun appel sortant
Asynchrone : consomme report.created, case.status-changed et media.added
Si panne : les événements attendent dans le broker et sont traités au redémarrage

MERCI

github.com/aymenRaddedi/ecosignal-SSA1